

Linguistica Formale e Modelli Computazionali

Anno accademico 2026-2027

Docente: Prof. Mario Bravetti

E-mail: mario.bravetti@unibo.it

Assistente: Dott. Lorenzo Bacchiani

E-mail: lorenzo.bacchiani2@unibo.it

Insegnamento da 6 crediti componente del corso integrato:

Ingegneria del Software e Modelli Computazionali

Orario lezioni:

- Mercoledì' 12 – 14, Laboratorio 2.2 e Aula 2.11, inizio effettivo ore 12 puntuali
- Giovedì' 10 – 13, Aula 2.1

Ricevimento

Su appuntamento da richiedere via e-mail

Comunicazioni e Materiale didattico (su Virtuale)

Linguistica formale

Linguaggi naturali/di programmazione: **lessico** e **sintassi**

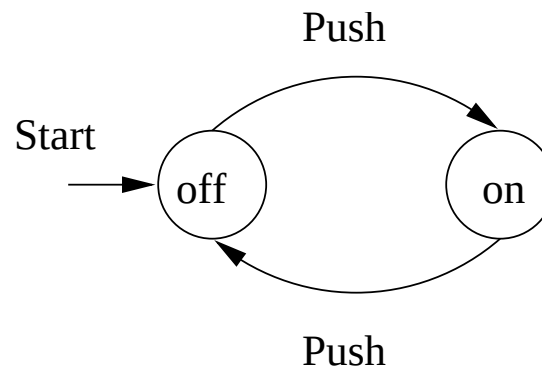
- Anni '50 studio dei linguaggi **naturali** (Noam Chomsky)
 - **Grammatiche** a struttura di frase (classificazione)
 - Sviluppo della teoria matematica relativa
- Inizio anni '60 applicazione ai linguaggi di **programmazione**
 - Linguaggio di programmazione: strumento per descrivere in modo rigoroso **procedure** per la risoluzione di problemi (es. funzioni che calcolano output a partire da un input)
 - Realizzazione **nuovi** linguaggi di programmazione e anche:
 - * domain-specific languages
 - * model-driven software development
 - * linguaggi logici (model checking)
 - * ...

Linguaggi di programmazione

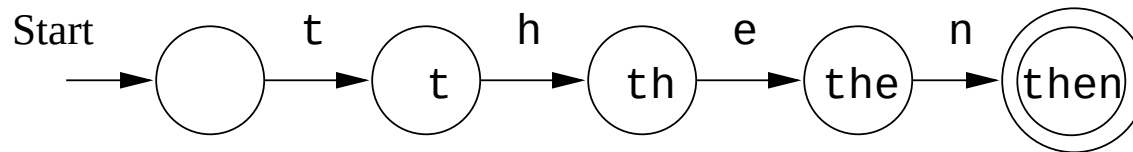
- **Compilatore:** front-end (parser, ...) + code generation
- Quanto e' complesso scrivere il codice di un compilatore?
 - Primo compilatore 1954–57 (Fortran) 18 anni-uomo
- **Generazione automatica** del codice del parser per un nuovo linguaggio tramite approccio dichiarativo
 - lessico: *espressioni regolari* (o automi a stati finiti)
 - sintassi: *grammatiche*, es. EBNF (o automi a pila)
- **Funzionamento** del parser generato:
 - input: sorgente di un programma
 - output:
 - * elenco errori lessicali/sintattici
 - * se privo di errori, **albero sintattico** del programma

Esempio di automi: automi a stati finiti

- Esempio: automa a stati finiti per un interruttore on/off



- Esempio: automa a stati finiti che riconosce la stringa then



Linguaggi naturali

- Frasi **ambigue**: data una frase in input, anche se corretta, il suo albero sintattico può **non essere unico**.
 - Esempio “I can fish”.
 - Due alberi sintattici possibili corrispondenti a:
“Io so pescare” / “Io inscatolo pesce”.
- **Grammatiche probabilistiche**: data una frase si può inferire l'**albero sintattico più probabile**.
- Esempio **classico** di (probabilistic) **Language Model (LM)**
 - Una sottoclasse dei (probabilistic) **Generative Model**
 - * modelli che rappresentano finitamente, tramite parametri, una **distribuzione di probabilità** su un certo **data space**
 - * il data space dei **LM** sono **sequenze di parole/token** (cioè frasi di un linguaggio naturale, come “I can fish”).

Language Model

Oltre a **grammatiche probabilistiche**, altri esempi **classici** di LM:

- **Bag Of Words** (in generale **n-gram**) **Model**
- **Hidden Markov Model**, cioè automi a stati finiti probabilistici (in generale Generative Model per sequenze di qualsiasi tipo)

Più recentemente LM basati su **modelli neurali**:

es. **Recurrent Neural Networks, LLM/Transformer**

- nell'approccio classico basato su **modelli probabilistici** molta conoscenza linguistica viene **esplicitamente** rappresentata (anche se i parametri probabilistici sono appresi da corpus/dati)
- nei **modelli neurali**, invece, le regolarità lessicali e sintattiche emergono **implicitamente** dai dati nell'apprendimento

Fronte della ricerca: modelli **neuro-simbolici**,

es. **grammatiche probabilistiche neurali**

Modelli computazionali

- In un **LM** è possibile fare **computazioni** tramite algoritmi:
 - **supervised/unsupervised learning**
(essendo modelli con parametri probabilistici)
 - **inference/classification** (parsing nelle grammatiche)
 - data **generation** tramite campionamento distribuzioni
- In questo senso gli LM, come **tutti i Generative Model**, sono **modelli computazionali**.
- Ma anche i **Discriminative Model**, alternativi ai Generative Model, sono modelli computazionali
 - **specializzati in classification** (no data generation)
 - es. **logistic regression, Convolutional Neural Networks**
- Anche altri: es. modelli per rappresentare **sistemi distribuiti**

Logiche di base e loro estensioni

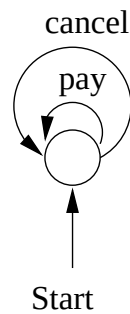
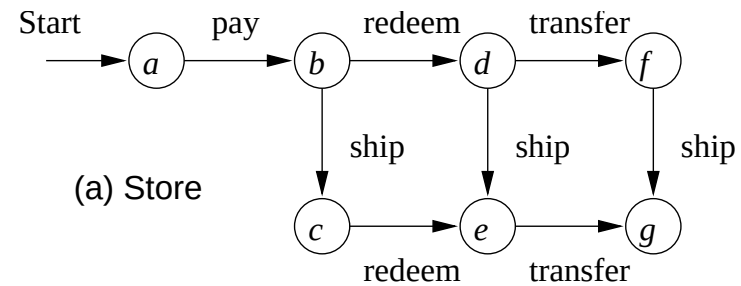
- Logica delle **Propositioni** e dei **Predicati**
- **Sistemi deduttivi** (assiomi e regole di inferenza) per costruire conseguenze logiche
- Noi vedremo estensione temporale: **Linear Time Logic**
- Ad es. utili per:
 - rappresentare la **conoscenza negli agenti intelligenti** (fondamenti matematici)
 - rappresentare la **semantica dei linguaggi naturali**
 - rappresentare e verificare (model checking) **proprietà di sistemi distribuiti**, es. safety, liveness, fairness.

Sistemi Distribuiti e Model Checking

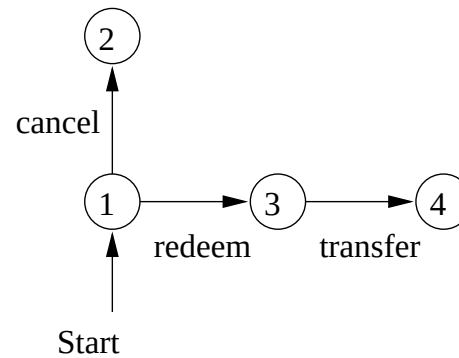
- Rappresentazione di sistemi **distribuiti/concorrenti**:
multipli programmi comunicanti tra loro
- **Model Checking**: stabilire alitmicamente se un sistema **soddisfa** (o meno) una data **proprietà** di **logica** temporale
- Modelli computazionali: **Labeled Transition System (LTS)**, **Process Algebra** e **Petri Net**

Commercio elettronico

Il protocollo (LTS) per **ogni partecipante**:

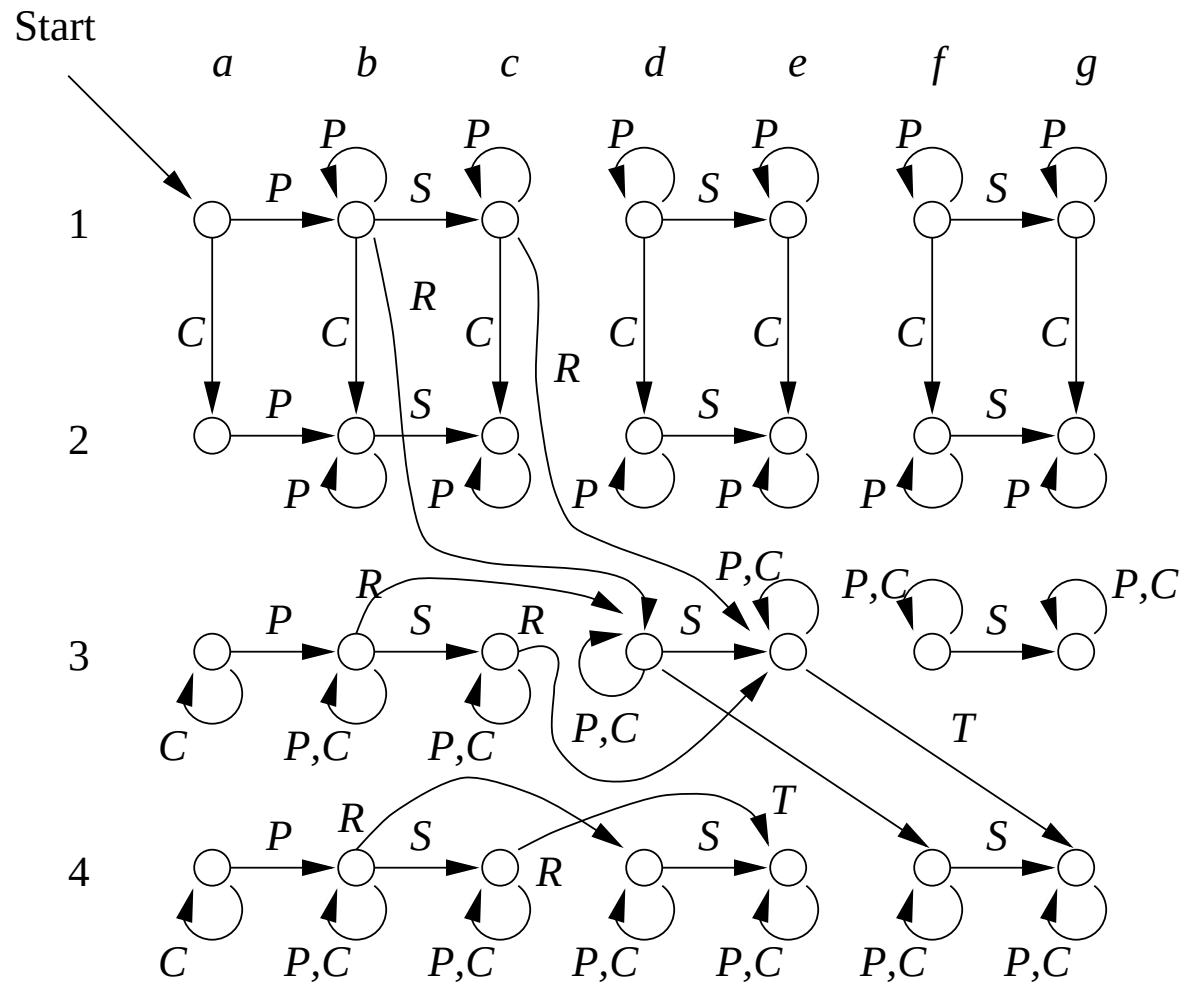


(b) Customer



(c) Bank

L'intero sistema come automa:



Logiche temporali

Permettono di esprimere **proprietà** del tipo:

non è possibile raggiungere uno stato in cui:

- *il negozio ha spedito (ship) e*
- *non è possibile raggiungere il trasferimento del denaro (transfer)*

Model checker: Strumenti che verificano se **proprietà espresse tramite logiche temporali valgono oppure no.**

Se non valgono forniscono **controesempi** (computazioni che non soddisfano la proprietà)

Contenuti del corso

- **Linguaggi formali e Automi**

Automati a stati finiti, espressioni regolari, grammatiche libere dal contesto e algoritmo Cocke-Younger-Kasami, automi a pila, classificazione di Chomsky delle grammatiche.

- **Applicazioni alla linguistica formale**

Grammatiche regolari e libere dal contesto probabilistiche, Hidden Markov Models, varianti probabilistiche dell'algoritmo Cocke-Younger-Kasami (Viterbi).

- **Logica di Base**

Logica proposizionale, sistemi deduttivi (regole di inferenza), logica dei predicati.

- **Sistemi Distribuiti e Model Checking**

Sistemi di transizione per la modellazione di sistemi distribuiti/concorrenti, logiche temporali per la specifica e verifica di proprietà dei sistemi, algebre di processi (processi concorrenti a stati finiti con sincronizzazione e interleaving), reti di Petri.

Laboratorio

Parte essenziale e integrante delle lezioni frontali del corso:

- **Supporto alla teoria** usando tool specifici.
 - **primo periodo** JFLAP 7.1: <http://www.jflap.org>
(automi/grammatiche/espressioni regolari)
 - Tina 3.8.0: <http://projects.laas.fr/tina>
(model checking di sistemi di transizione e reti di Petri)
 - LTSA 3.0: <http://www.doc.ic.ac.uk/ltsa>
(sistemi di transizione definiti tramite algebre di processi)
- **Installare** subito JFLAP 7.1 sui propri PC/portatili al fine di:
 - **esercitarsi a casa** (verranno dati di volta in volta esercizi da fare per la settimana successiva) e
 - se si usa il proprio portatile, effettuare le **esercitazioni in laboratorio**

Libri di testo

- J. E. Hopcroft, R. Motwani e J. D. Ullman:
Automi, linguaggi e calcolabilità,
Pearson Education Italia S.R.L, Terza Edizione, 2021.
Cap. 1–9
(disponibile online per studenti UniBO)
- S. J. Russell, P. Norvig:
Artificial Intelligence: A Modern Approach,
Pearson International, 4th edition, Global Edition, 2022.
Principalmente Cap. 24
(disponibile online per studenti UniBO)
- M. Huth e M. Ryan:
Logic in Computer Science: Modelling and Reasoning about Systems,
Cambridge University Press, Second Edition, 2004.
Cap. 1–3

Esame

Scritto:

3 appelli sessione invernale (Gennaio e Febbraio),
3 appelli sessione estiva (Giugno, Luglio e Settembre).

Quesiti di due tipi:

- **domande** aventi lo scopo di verificare l'acquisizione delle competenze teoriche del corso;
- **esercizi** aventi lo scopo di verificare la capacità di applicazione dei concetti teorici su semplici casi di studio.

Voto in 30esimi

che **fa media** con il voto dell'altro componente del corso integrato:
insegnamento Advanced Software Engineering And Development